



**Nass Merchant Payment Gateway
Integration Manual
v1.3.0**

Introduction	3
Integration Methods	3
1. Using Nass Portal	3
1.1 Base URLs	3
1.2 Authentication	4
1.3 Transaction Processing	5
1.4 Callbacks	7
1.5 UAT Test Cards	8
1.6 Status Checks.....	8
2. Using Direct API Integration	9
2.1 Base URLs	9
2.2 Transaction Flow.....	9
2.3 Required Parameters for Direct API Transaction	12
Error Handling	13

Introduction

The Nass Merchant Payment Gateway allows merchants to securely process transactions through a REST API. This document provides a comprehensive, step-by-step guide for integrating the gateway, including details on authentication and transaction initiation.

Integration Methods

Merchants can integrate with the Nass Payment Gateway using one of the following methods:

Nass Payment Gateway Portal – A user-friendly web interface for processing payments and managing transactions.

Direct API Integration – Suitable for merchants who prefer direct interaction with the API for greater control over transaction processing.

1. Using Nass Portal

1.1 Base URLs

- **UAT Environment:**

<https://uat-gateway.nass.iq:9746/>

- **Production Environment:**

The base URL will be provided by your account manager or the support team.

1.2 Authentication

Before initiating transactions, merchants must authenticate using their credentials.

- **Endpoint:** /auth/merchant/login
- **Method:** POST
- **Description:** Authenticates the merchant and returns a session token.

Request Body:

```
{  
  "username": "merchant_username",  
  "password": "merchant_password"  
}
```

Response (200 OK):

```
{  
  "access_token": "your_access_token"  
}
```

Note: The access token must be included in all subsequent requests in the Authorization header using the following format:

Bearer <token>

1.3 Transaction Processing

Once authenticated, merchants can initiate transactions.

- **Endpoint:** /transaction
- **Method:** POST
- **Description:** Creates a new payment transaction.

Request Body:

```
{  
  "orderId": "123456",  
  "orderDesc": "Purchase of electronics",  
  "amount": 150.00,  
  "currency": "368",  
  "transactionType": "1",  
  "backRef": "<Frontend Redirect URL>",  
  "notifyUrl": "<HTTP Backend URL for sending POS request callbacks>"  
}
```

Response (201 Created)

```
{
  "success": true,
  "code": 0,
  "status_code": 200,
  "data": {
    "url": "https://3dsecure.nass.iq/gateway/{Transaction Parameters..}",
    "pSign": "18f...",
    "transactionParams": {
      "TERMINAL": "<TERMINAL_ID>",
      "TRTYPE": "1",
      "AMOUNT": "100",
      "TIMESTAMP": "20250406073541",
      "NONCE": "cc27f848a80822cb1d7ff618e35554a7",
      "CURRENCY": "368",
      "ORDER": "123456",
      "DESC": "Purchase of electronics",
      "MERCH_NAME": "demo",
      "MERCH_URL": "",
      "EMAIL": "demo@nass.iq",
      "COUNTRY": "IRAQ",
      "MERCH_GMT": "Asia/Baghdad",
      "BACKREF": "https://<provided-by-merchnat>"
    }
  }
}
```

Note: To complete the payment, merchants must redirect customers to the data.url returned in the response, along with the provided parameters.

1.4 Callbacks

Merchants should include a callback URL in the initial request. This URL will be used by the payment system to send the transaction response back to the merchant's system. All responses received at the callback URL should be securely stored for future reference.

- **Method:** POST
- **Encoding:** application/json

Response Schema:

```
{
  "terminal": "<NUMBER>",
  "actionCode": "0",
  "responseCode": "00", // success
  "statusMsg": "Approved",
  "card": "4761XXXXXXXXX0047",
  "amount": "5000",
  "currency": "368",
  "tranDate": "2025.02.27 17:41:35",
  "rrn": "505801383493", // Transaction Reference Number in Acquiring Bank
  "intRef": "F1A9E559A687E038", // Transaction Reference Number for Reverse and
  "nonce": "cd236bf9280d26d5d16813ffdce1db96",
  "signature": "7D5...",
  "orderId": "123456",
  "timestamp": "20250227144135"
}
```

1.5 UAT Test Cards

The following card details are for **testing purposes only** and may be used exclusively in the UAT environment.

The following test cards are available **for use exclusively in the UAT environment**:

PAN	Expiry	CVV
5185520050000010	12/25	356
4761349999000039	12/31	Not Provided cvc2_rc = 0

1.6 Status Checks

Merchants can check the status of a transaction within 24 hours of its initiation.

- **URL:** /transaction/\${OrderId}/checkStatus
- **Method:** GET
- **Description:** Retrieves the transaction status.

Response Schema:

```
{
  "success": true,
  "code": 0,
  "status_code": 200,
  "data": {
    "terminal": "00053402",
    "actionCode": "0",
    "responseCode": "00",
    "statusMsg": "Approved",
    "card": "4747XXXXXXXXX1233",
    "amount": "500",
    "currency": "368",
    "tranDate": "2025.03.01 13:46:25",
    "rrn": "506001398324",
    "intRef": "35E022E154BEDD46",
    "nonce": "1605cf16a7e580b6ae7f1b078d45b635",
    "signature": "0EB...",
    "timestamp": "20250301104625"
  }
}
```


2. Using Direct API Integration

Direct API Integration allows merchants to process transactions by interacting directly with the Nass Payment Gateway API, bypassing the portal. This method requires merchants to securely transmit cardholder data.

2.1 Base URLs

- **UAT Environment:** <https://uat-gateway.nass.iq:9746/>
- **Transaction Processing URL (tURL):** https://3dsecure.nass.iq/cgi-bin/cgi_json

2.2 Transaction Flow

1. Authentication

- a. Merchants must authenticate using their credentials by sending a POST request to the /auth/merchant/login endpoint.
- b. Upon successful authentication, the API returns an access token, which must be included in the Authorization header of all subsequent requests using the format:
Bearer <access_token>

2. Initiating a Transaction

- a. Send a POST request to /transaction to initiate a transaction.
- b. The request body must include details such as orderId, amount, currency, and transactionType.
- c. The response will contain pSign and transactionParams (including nonce and timestamp), which are required for Direct API Integration.

3. Sending Cardholder Data for Authorization

- a. After receiving the response from /transaction, the merchant system must send a POST request to the transaction processing URL (tURL).
- b. **The request should include the following:**
 - pSign: The signature from the initial transaction response
 - NONCE: A unique identifier for the transaction
 - Cardholder details (PAN, expiry date, CVV).
 - Order Number, Amount and all other parameters from the initial request.

c. **Example Request Body:**

```
d. {
e.   "card": "4761349999000047",
f.   "expMonth": "12",
   "expYear": "30",
   "currency": "368",
   "name": "John Doe",
   "trtype": "1",
   "terminal": "00053402",
   "desc": "NOKIA 3310",
   "cvc2_rc": "0",
   "orderId": "000009",
   "timestamp": "20250302101621",
   "nonce": "a6ddb5e5ab7908ff9b347382cd07a3d3",
   "p_sign":
   "9083f8675a3cde012a10e7eb53032dd967df71837f52830f109c8c7ced081c32f9009c49fda6284c778
0998824eb83ba25d8f9768778a3e32c1f4949d9a33020458ef07664f90f273d422bb4de90e209bb462
81b7c8f8d36286b33e75073e305acab8bf9a828005ff178c1073c7a29e2191c467ef67cc1d64bc301fbfd
3c2a9e11c1c6a2a003071141d3bcdcc67bc72bbdf2f1cfb44f6ca95e2b98db85e73d75a873ae3a15b800
18141328dd7058cd9be30223d385c4e4f7d9deae779caa16f6d4efccff8adfb070fe247cdfbf16b4e048b3
d61ef67acfcc6b649c21c65689ddd8424d8dc7fc90daa45de71cec8736423b21ff613d98fc0d606b1279
8a99b904",
   "amount": "23000",
   "cvc2": "356",
   "backRefUrl": "https://www.sample.com/shop/reply"
}
```

4. **Receiving Authorization Response**

- a. If successful or failed, the system will send back an html which if it's rendered it will directly redirect back to the merchant website
- b. In the window.location.href it will be redirected to the merchants website appending status,orderId and the RRN of the transaction
- c. **Example Response template:**

```
i.
<!DOCTYPE html>
<html>

<head>
<title>Redirecting..</title>
<meta http-equiv="Pragma" content="no-cache">
<meta http-equiv="Cache-Control" content="no-store">
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<script>
window.onload = function() {
// Simple redirect to the URL in https://<backRefUrl provided in request body> placeholder
window.location.href = https://<backRefUrl provided in request
body>?status=success&orderId=13265359&amount=1000.000&rrn=509601682074; // parameters of status orderId
and rrn will be appended to the full url as query param.
```

```
}  
</script>  
</head>  
  
<body>  
<h2>Redirecting, please wait...</h2>  
</body>  
  
</html>
```

5. Handling Callbacks

- Merchants must provide a callback URL in the transaction request. This can be done by including the parameter "backRefUrl": "<Callback Endpoint>" in the request body for a dynamic callback specific to that transaction.
- If this parameter is not provided, the Nass Payment Gateway will send the transaction result to the default callback URL configured for the merchant account.

6. Checking Transaction Status

- Merchants can check the status of a transaction within **24 hours** of initiation by sending a **GET** request to the following endpoint:

GET \${baseUrl}/transaction/\${OrderId}/checkStatus

2.3 Required Parameters for Direct API Transaction

Name	Data Type	M/ O/C	Description
amount	Numeric	M	Order total amount.
backRefUrl	String	O	Merchant URL to post the result message.
card	Numeric	M	Card Number (PAN).
currency	String	M	Order currency (3-character ISO currency code).
cvc2	String	M	Card CVV2/CVC2 security code.
desc	String	M	Order description.
expMonth	String	M	Card expiration month (MM format).
expYear	String	M	Card expiration year (YY format).
orderId	Numeric	M	Unique order identifier generated by the merchant.
terminal	String	M	Merchant Terminal ID assigned by the bank.
timestamp	String (YYYYMMDDHHMMSS)	M	Merchant transaction timestamp in GMT.
p_sign	String	M	Generated mac information for this transaction.
nonce	String	M	Randon random bytes in hexadecimal format. Must be present if MAC is used.
CVC2_RC	String	O	CVC2 reason code. Below values used 1 for CVC2 is present 0 for CVC2 is not provided 2 for CVC2 is illegible 9 for No CVC2 on card

Legend:

- **M** = Mandatory
- **O** = Optional
- **C** = Conditional

Error Handling

Common Error Responses

- **400 Bad Request:** Missing or invalid parameters.
- **401 Unauthorized:** Invalid authentication credentials.
- **500 Internal Server Error:** Unexpected server issue.

Transaction Error Codes:

Response Code	Description
-1	A mandatory field in the request is not filled in
-2	CGI request validation failed
-3	Acquirer host (TS) is not responding or invalid format of e-Gateway response template file
-4	No connection to the acquirer host (TS)
-5	The acquirer host (TS) connection failed during transaction processing
-6	e-Gateway configuration error
-7	The acquirer host (TS) response is invalid, e.g. mandatory fields missing
-8	Error in the request's "Card number" field
-9	Error in the request's "Card expiration date" field
-10	Error in the request's "Amount" field
-11	Error in the request's "Currency" field
-12	Error in the request's "Merchant ID" field
-13	The IP address of the transaction source (usually the merchant's IP) is not the one expected
-14	No connection to the internet terminal PIN pad or agent program is not running on the internet terminal computer/workstation
-15	Error in the request's "RRN" field
-16	Another transaction is being performed on the terminal
-17	The terminal is denied access to e-Gateway
-18	Error in the request's "CVC2" or "CVC2 Description" fields
-19	Error in the authentication information request or authentication failed
-20	The allowed time interval (1 hour by default) between the request's transaction "Time Stamp" field and the e-Gateway time was exceeded
-21	The transaction has already been executed
-22	Transaction contains invalid authentication information
-23	Invalid transaction context
-24	Transaction context data mismatch
-25	Transaction cancelled by user

-26	Invalid card BIN
-27	Invalid merchant name
-28	Invalid addendum(s)
-29	Invalid/duplicate authentication reference
-30	Transaction was declined as fraud
-31	Transaction already in progress
-32	Repeated declined transaction
-33	Customer authentication by random amount or verify one-time code in progress
-34	Mastercard Installment customer choice in progress
-35	Mastercard Installments auto canceled
-36	Mastercard Installment user canceled
-37	Error in request's recurring payment expiry date field
-38	UPI (China Union Pay) server error message
-39	Transaction in the process of confirmation
-40	Card data specified by a customer in the form is being processed
-41	PAReq error when a request cannot be processed without an enrollment status check (MPASS Wallet)
-42	MPASS Wallet: PAReq is required before continuing the transaction
-43	3D-Secure authentication attempt
-44	3D-Secure authentication is not available
-45	Card is not enrolled in 3D-Secure authentication program
-46	The error is not critical, the transaction may be continued/repeated
-47	UPI transaction is being processed

For more information, please refer to the full API documentation or contact the support team for assistance.